

FutureKawa

Base de Données — Application Siège

Équipe : Rudy · Florian · Noé · Josselin

Version : 1.0

Date : 02/07/2026

Projet : FutureKawa

Sommaire

Base de Données Siège — FutureKawa

01 — Vue d'ensemble

02 — Utilisateurs & authentification

03 — Token de session DRF

04 — Sites d'exploitation

05 — Relations

06 — Dictionnaire de données

Base de Données Siège — FutureKawa

Projet	FutureKawa
Document	Documentation de la base de données siège
Version	1.0
Date	02/07/2026
Équipe	Rudy · Florian · Noé · Josselin

Résumé

La base de données siège contient **4 tables** qui couvrent deux responsabilités distinctes : la gestion des utilisateurs de l'interface centrale et le référentiel des plantations supervisées.

Table	Rôle	Lignes estimées
<code>SIEGE_USER</code>	Comptes utilisateurs (AbstractUser Django)	< 100
<code>INVITATION_TOKEN</code>	Tokens d'invitation à usage unique (24 h)	< 100
<code>authtoken_token</code>	Tokens de session API (DRF)	= nombre d'utilisateurs actifs
<code>EXPLOITATION_SITE</code>	Référentiel des plantations + cache JSON	< 50

Navigation

#	Document	Contenu
1	Vue d'ensemble	Rôle de la BDD, technologie, conventions
2	Utilisateurs & authentification	SIEGE_USER, INVITATION_TOKEN, flux d'invitation
3	Token de session DRF	authtoken_token, cycle de vie
4	Sites d'exploitation	EXPLOITATION_SITE, cache JSON, mécanisme de sync
5	Relations	Schéma des FK, conventions, résumé
6	Dictionnaire de données	Référence complète de toutes les colonnes

01 — Vue d'ensemble

Rôle de la base de données siège

La base de données siège est le **référentiel central** de l'application FutureKawa côté siège. Elle ne contient pas de données de plantation (mesures, lots, alertes) — celles-ci restent dans chaque instance d'exploitation. Son rôle est double :

- 1. **Gérer les comptes utilisateurs** de l'interface siège et leur accès
- 2. **Centraliser la liste des plantations** supervisées et stocker un snapshot de leurs données pour la résilience réseau

Technologie

Élément	Choix	Justification
SGBD	PostgreSQL 15	Fiabilité, support JSON natif (pour <code>last_data</code>)
ORM	Django ORM (migrations)	Gestion des schémas versionnée, intégrée au framework
Authentification	Django REST Framework <code>authtoken</code>	Token stateful simple, adapté à une API interne

Architecture dans le projet



| HTTP POST /sites/{id}/sync/



[API Exploitation distante]

└─ répond avec /api/all-data/ (JSON)

→ stocké dans `EXPLOITATION_SITE.last_data`

La BDD siège est isolée de la BDD exploitation : aucune FK croisée, aucune réplication. La synchronisation est **applicative** via HTTP.

Conventions de modélisation

Convention	Valeur
Clé primaire	<code>id</code> entier auto-incrémenté (sauf <code>authtoken_token.key</code>)
Horodatages	<code>created_at</code> / <code>updated_at</code> auto-gérés par Django
FK obligatoire	<code>CASCADE</code> par défaut
FK nullable	<code>SET_NULL</code> , indiqué avec <code>null=True</code> , <code>blank=True</code>
UUID	Utilisé uniquement pour <code>INVITATION_TOKEN.token</code>

Schéma de référence

Fichier	Format	Usage
bdd-siege/schema-siege.png	PNG	Visualisation rapide
bdd-siege/schema.puml	PlantUML	Source éditable

02 — Utilisateurs & authentication

Table SIEGE_USER

Étend `AbstractUser` de Django. Chaque compte appartient à un rôle qui détermine les droits dans l'interface siège.

Colonne	Type	Contrainte	Description
<code>id</code>	int	PK, auto	Identifiant unique
<code>username</code>	varchar(150)	UNIQUE, NOT NULL	Identifiant de connexion
<code>email</code>	varchar(254)	NOT NULL	Adresse e-mail
<code>first_name</code>	varchar	—	Prénom
<code>last_name</code>	varchar	—	Nom
<code>password</code>	varchar	NOT NULL	Hash du mot de passe (bcrypt via Django)
<code>role</code>	varchar(20)	NOT NULL	Rôle applicatif (<code>admin</code> , <code>manager</code> , <code>viewer</code>)
<code>is_active</code>	bool	DEFAULT true	Compte actif ou désactivé
<code>is_staff</code>	bool	DEFAULT false	Accès à l'interface d'administration Django

Rôles

Rôle	Accès
<code>admin</code>	Gestion des utilisateurs, des sites, de tous les paramètres

Rôle	Accès
manager	Lecture de toutes les données, synchronisation des sites
viewer	Lecture seule — tableau de bord et détails des sites

Table INVITATION_TOKEN

Chaque nouvel utilisateur reçoit un token UUID par e-mail pour définir son mot de passe. Le token est **à usage unique** et expire après **24 heures**.

Colonne	Type	Contrainte	Description
id	int	PK, auto	Identifiant
user_id	int	FK → SIEGE_USER, CASCADE	Utilisateur concerné
token	uuid	UNIQUE, NOT NULL	Jeton généré à la création du compte
expires_at	datetime	NOT NULL	Date d'expiration (création + 24 h)

Flux d'invitation

Le flux d'invitation permet de créer un compte sans que l'administrateur connaisse le mot de passe de l'utilisateur.

- Admin `POST /api/users/`
 - SIEGE_USER créé (`is_active=False`, `password` vide)
 - INVITATION_TOKEN créé (`uuid`, `expires_at = now + 24h`)
 - E-mail envoyé : `https://siege.futurekawa.fr/set-password?token=<uuid>`
- Utilisateur `GET /set-password?token=<uuid>`
 - API vérifie : `token` existe + non expiré
- Utilisateur `POST /api/auth/set-password/` { `token`, `password` }

- Mot de passe hashé + is_active=True
- INVITATION_TOKEN supprimé (usage unique)
- SIEGE_USER actif, connexion possible

Cas d'erreur

Situation	Comportement
Token expiré (<code>expires_at</code> < now)	400 — « Lien expiré, demandez un nouvel envoi »
Token inexistant	404 — « Lien invalide »
Token déjà utilisé (supprimé)	404 — « Lien invalide »
Mot de passe ne respecte pas la politique	400 — validation Django

Ré-invitation

Un administrateur peut renvoyer un e-mail d'invitation en supprimant l'ancien token et en en créant un nouveau via

`POST /api/users/{id}/resend-invitation/` .

03 — Token de session DRF

Table authtoken_token

Table gérée automatiquement par `rest_framework.authtoken` (Django REST Framework). Elle stocke le token de session utilisé pour authentifier les requêtes API après connexion.

Colonne	Type	Contrainte	Description
<code>key</code>	<code>varchar(40)</code>	PK	Token hexadécimal généré automatiquement
<code>user_id</code>	<code>int</code>	FK → SIEGE_USER, CASCADE, UNIQUE	Utilisateur propriétaire du token
<code>created</code>	<code>datetime</code>	NOT NULL	Date de création

Fonctionnement

Connexion

Lors d'un `POST /api/auth/login/` avec des identifiants valides : 1. DRF vérifie les credentials 2. Un token est créé ou récupéré dans `authtoken_token` 3. Le token est retourné dans la réponse (et stocké dans un cookie `HttpOnly` côté frontend)

```
POST /api/auth/login/
{ "username": "rudy", "password": "...." }

→ 200 OK
→ Set-Cookie: auth_token=abc123def456...; HttpOnly; Secure
```

Authentification des requêtes

Toutes les requêtes authentifiées envoient le token dans le header :

```
Authorization: Token abc123def456...
```

Le middleware DRF vérifie la table `authtoken_token` à chaque requête.

Déconnexion

`POST /api/auth/logout/` supprime l'entrée dans `authtoken_token`. Le token est donc invalidé côté serveur — il ne peut plus être utilisé même s'il est encore présent dans un cookie.

Différence avec INVITATION_TOKEN

	<code>authtoken_token</code>	<code>INVITATION_TOKEN</code>
Rôle	Authentification de session	Première définition du mot de passe
Durée de vie	Tant que l'utilisateur ne se déconnecte pas	24 h, usage unique
Géré par	Django REST Framework	Code applicatif custom
Supprimé quand	Déconnexion explicite	Utilisation du lien
Format	Hex 40 chars	UUID

Considérations de sécurité

- Le token DRF est un **token stateful** : sa révocation côté serveur est immédiate.
- La table `authtoken_token` impose `UNIQUE` sur `user_id` : un utilisateur ne peut avoir qu'un seul token actif à la fois.
- Le cookie est défini avec `HttpOnly` et `Secure` pour prévenir les attaques XSS et forcer HTTPS.

04 — Sites d'exploitation

Table EXPLOITATION_SITE

Référentiel des plantations supervisées depuis le siège. Chaque entrée pointe vers une instance d'API exploitation distante et conserve un **snapshot JSON** de ses dernières données connues.

Colonne	Type	Contrainte	Description
id	int	PK, auto	Identifiant unique
name	varchar(255)	NOT NULL	Nom de la plantation
location	varchar(255)	NOT NULL	Localisation textuelle
country	varchar(100)	NOT NULL	Pays
api_url	varchar(500)	NOT NULL	URL de base de l'API exploitation distante
api_token	varchar(500)	NOT NULL	Master token d'accès à l'API distante
status	varchar(20)	NOT NULL	Statut de connectivité (<code>online</code> , <code>offline</code> , <code>unknown</code>)
image_url	varchar(500)	—	Image illustrative de la plantation
last_data	json	—	Snapshot JSON du dernier <code>/api/all-data/</code> réussi

Colonne	Type	Contrainte	Description
last_sync	datetime	—	Horodatage de la dernière synchronisation réussie
created_at	datetime	auto	Date de création de l'entrée
updated_at	datetime	auto	Date de dernière modification

Mécanisme de synchronisation (tampon)

Le siège ne contacte pas l'API exploitation en temps réel à chaque requête utilisateur. Il maintient un **cache local** (`last_data`) mis à jour à la demande.

Déclenchement

```
POST /api/sites/{id}/sync/
```

1. L'API siège contacte : GET {api_url}/api/all-data/
avec le header : Authorization: Token {api_token}

2a. Succès :

- last_data = réponse JSON complète
- last_sync = now()
- status = "online"
- réponse : { from_cache: false, ...données }

2b. Erreur réseau / timeout :

- last_data inchangé
- status = "offline"
- réponse : { from_cache: true, ...last_data }

Structure de last_data

```
{
  "warehouses": [
    { "id": 1, "name": "Entrepôt A", "surface_m2": 500.0 }
  ],
  "measures": [
    { "id": 42, "sensor_id": 3, "temperature": 22.1, "humidity":
64.5, "measure_date": "2026-07-02T10:00:00Z" }
  ],
  "batches": [
    { "id": "uuid...", "variety": "Arabica", "status": "active" }
  ],
  "alerts": [
    { "id": 7, "message": "Température trop haute", "resolved":
false }
  ],
  "movements": [
    { "warehouse_id": 1, "batch_id": "uuid...", "movement_type":
"in" }
  ],
  "stats": {
    "active_batches": 12,
    "unresolved_alerts": 2,
    "total_warehouses": 3
  }
}
```

Statuts de connectivité

Statut	Signification	Comportement frontend
online	Dernière sync réussie	Indicateur vert, données fraîches
offline	Dernière sync échouée	Indicateur rouge, données en cache affichées

Statut	Signification	Comportement frontend
unknown	Jamais synchronisé	Indicateur gris, bouton « Synchroniser »

Sécurité du api_token

Le `api_token` stocké dans `EXPLOITATION_SITE` est le **master token** de l'instance exploitation. Il donne accès à toutes les données via `/api/all-data/`. Il est : - **Chiffré en base de données** avec Fernet (chiffrement symétrique AES-128-CBC) — la clé de chiffrement est fournie via la variable d'environnement `SITE_TOKEN_ENCRYPTION_KEY`, jamais codée en dur en production - **Jamais retourné dans les réponses API** (champ `write_only` côté sérialiseur DRF) — un GET sur `/api/sites/` ne renvoie pas le token - Révocable depuis l'interface administration de l'exploitation via `manage.py create_master_token`

Générer la clé de chiffrement (production)

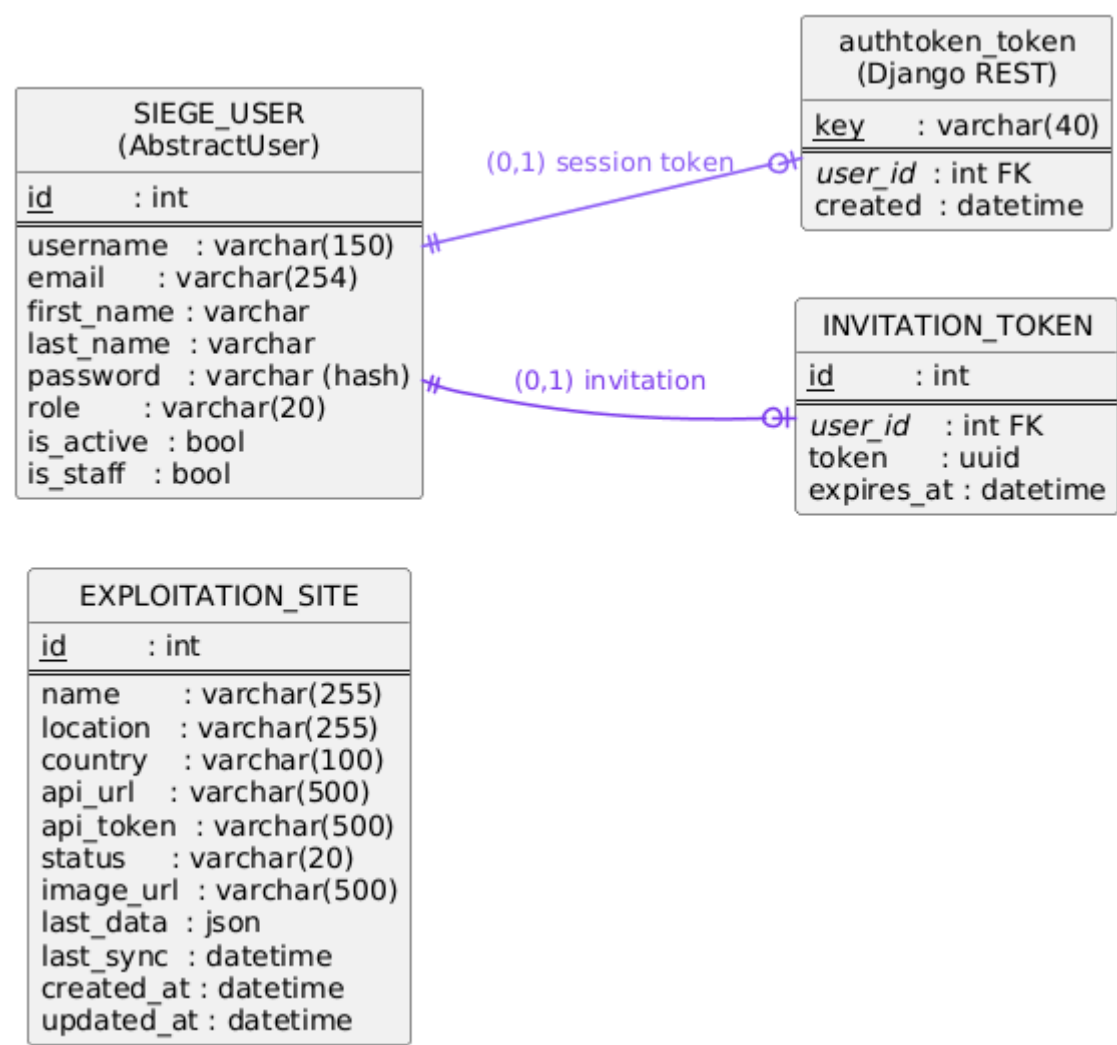
```
python -c "from cryptography.fernet import Fernet;
print(Fernet.generate_key().decode())"
```

Définir ensuite dans `.env` :

```
SITE_TOKEN_ENCRYPTION_KEY=<clé générée>
```

05 — Relations

Schéma des relations



Détail des clés étrangères

Table source	Colonne	Table cible	Colonne	On Delete
INVITATION_TOKEN	user_id	SIEGE_USER	id	CASCADE
authtoken_token	user_id	SIEGE_USER	id	CASCADE

Cardinalités

Relation	Cardinalité	Note
SIEGE_USER → INVITATION_TOKEN	1 à 0..1	Au plus un token d'invitation en attente par utilisateur
SIEGE_USER → authtoken_token	1 à 0..1	Au plus un token de session (UNIQUE sur user_id)
SIEGE_USER → EXPLOITATION_SITE	sans FK	Un utilisateur accède aux sites via l'API, pas par FK

Tables indépendantes

`EXPLOITATION_SITE` n'a aucune FK vers `SIEGE_USER`. Tout utilisateur authentifié (quel que soit son rôle) peut consulter les sites — le contrôle d'accès est géré au niveau applicatif (vérification du rôle dans la vue DRF), pas au niveau base de données.

Conventions CASCADE

- **CASCADE** sur `INVITATION_TOKEN.user_id` : si un utilisateur est supprimé, son token d'invitation est supprimé en même temps. Évite les tokens orphelins.
- **CASCADE** sur `authtoken_token.user_id` : idem pour les tokens de session. La suppression d'un compte révoque immédiatement toutes ses sessions.

Résumé des tables

Table	PK	FK	Lignes	Géré par
<code>SIEGE_USER</code>	<code>id</code> (int)	—	< 100	Django AbstractUser
<code>INVITATION_TOKEN</code>	<code>id</code> (int)	<code>user_id</code>	= comptes sans mdp	Code custom

Table	PK	FK	Lignes	Géré par
authtoken_to ken	key (varchar)	user_id	= sessions actives	DRF authtoken
EXPLOITATION _SITE	id (int)	—	< 50	Code custom

06 — Dictionnaire de données

Référence complète de toutes les colonnes de la base de données siège, avec leur type PostgreSQL, contraintes et description métier.

Table : `siege_user`

Hérite du modèle `AbstractUser` de Django. Colonnes propres à FutureKawa en gras.

Colonne	Type PostgreSQL	Null	Défaut	Description
<code>id</code>	<code>bigint</code>	Non	<code>auto</code>	Clé primaire auto-incrémentée
<code>username</code>	<code>varchar(150)</code>	Non	—	Identifiant de connexion unique
<code>email</code>	<code>varchar(254)</code>	Non	—	Adresse email unique, utilisée comme contact
<code>role</code>	<code>varchar(20)</code>	Non	<code>'viewer'</code>	Rôle applicatif : <code>admin</code> ou <code>viewer</code>
<code>password</code>	<code>varchar(128)</code>	Non	—	Hash du mot de passe (Django PBKDF2) ou marqueur <code>!</code> (inutilisable)

Colonne	Type PostgreSQL	Null	Défaut	Description
<code>is_staff</code>	boolean	Non	<code>false</code>	Accès à l'interface Django admin — mis à <code>true</code> pour les admins
<code>is_active</code>	boolean	Non	<code>true</code>	Compte actif. <code>false</code> = compte désactivé sans suppression
<code>is_superuser</code>	boolean	Non	<code>false</code>	Permissions totales sans restriction
<code>first_name</code>	varchar(150)	Non	<code>''</code>	Prénom (optionnel, non affiché dans l'interface)
<code>last_name</code>	varchar(150)	Non	<code>''</code>	Nom (optionnel)
<code>date_joined</code>	timestampz	Non	<code>now()</code>	Date de création du compte
<code>last_login</code>	timestampz	Oui	<code>null</code>	Dernière connexion

Table : invitation_token

Token à usage unique (UUID) envoyé par email au nouvel utilisateur pour définir son mot de passe.

Colonne	Type PostgreSQL	Null	Défaut	Description
<code>id</code>	bigint	Non	auto	Clé primaire

Colonne	Type PostgreSQL	Null	Défaut	Description
<code>user_id</code>	<code>bigint</code>	Non	—	FK → <code>siege_user.id</code> (CASCADE) — un token par utilisateur max
<code>token</code>	<code>uuid</code>	Non	<code>uuid4()</code>	Token généré aléatoirement, transmis dans le lien d'invitation
<code>expires_at</code>	<code>timestampz</code>	Non	—	Date d'expiration (24 h après création)

Contraintes : - `UNIQUE` sur `user_id` (relation 1-à-1 via `OneToOneField`) - `UNIQUE` sur `token`

Table : `authtoken_token`

Gérée par `rest_framework.authtoken`. Token de session envoyé dans chaque requête API.

Colonne	Type PostgreSQL	Null	Défaut	Description
<code>key</code>	<code>varchar(40)</code>	Non	—	Clé primaire — token hexadécimal de 40 caractères
<code>user_id</code>	<code>bigint</code>	Non	—	FK → <code>siege_user.id</code> (CASCADE) — un token par utilisateur

Colonne	Type PostgreSQL	Null	Défaut	Description
<code>created</code>	timestampztz	Non	—	Date de création du token

Contraintes : - `UNIQUE` sur `user_id` — un utilisateur ne peut avoir qu'un seul token actif

Table : exploitation_site

Référentiel des plantations supervisées. Stocke les coordonnées d'accès à chaque API exploitation et le cache de ses dernières données.

Colonne	Type PostgreSQL	Null	Défaut	Description
<code>id</code>	bigint	Non	auto	Clé primaire
<code>name</code>	varchar(255)	Non	—	Nom de la plantation
<code>location</code>	varchar(255)	Non	<code>''</code>	Localisation textuelle (ville, région...)
<code>country</code>	varchar(100)	Non	—	Pays
<code>api_url</code>	varchar(500)	Non	—	URL racine de l'API exploitation distante
<code>api_token</code>	text	Non	—	Master token d'accès — stocké chiffré (Fernet/AES-128)

Colonne	Type PostgreSQL	Null	Défaut	Description
status	varchar(20)	Non	'unknown'	Connectivité : unknown, active, error, unreachable
image_url	varchar(500)	Non	''	URL d'une image illustrant la plantation (optionnel)
last_data	jsonb	Oui	null	Snapshot du dernier /api/all-data/ réussi
last_sync	timestamptz	Oui	null	Horodatage de la dernière synchronisation réussie
created_at	timestamptz	Non	now()	Date de création de l'entrée
updated_at	timestamptz	Non	now()	Date de dernière modification

Récapitulatif des types

Type Django	Type PostgreSQL	Remarque
BigAutoField	bigint	PK auto-incrémentée
CharField(max_length=N)	varchar(N)	Chaîne limitée
TextField	text	Chaîne illimitée (utilisée pour api_token chiffré)

Type Django	Type PostgreSQL	Remarque
EmailField	varchar(254)	VARCHAR avec validation Django
BooleanField	boolean	
DateTimeField	timestampz	Toujours avec fuseau horaire (Europe/Paris)
UUIDField	uuid	
JSONField	jsonb	JSON binaire indexable
OneToOneField / ForeignKey	bigint + contrainte FK	